End-to-End ETL Pipeline Project Documentation

1. Project Overview

This project implements a complete End-to-End ETL (Extract, Transform, Load) pipeline designed to automate data acquisition, processing, and storage for downstream analytics. The system leverages Python for scripting, MySQL for structured storage, and GitHub Actions for continuous integration and scheduled automation. The pipeline follows a multi-layered approach consisting of raw data extraction, transformation logic, and ingestion into a database, ensuring scalability, repeatability, and reliability. This document elaborates on the project's architecture, objectives, methodology, workflow automation, advantages, limitations, and final expected outputs.

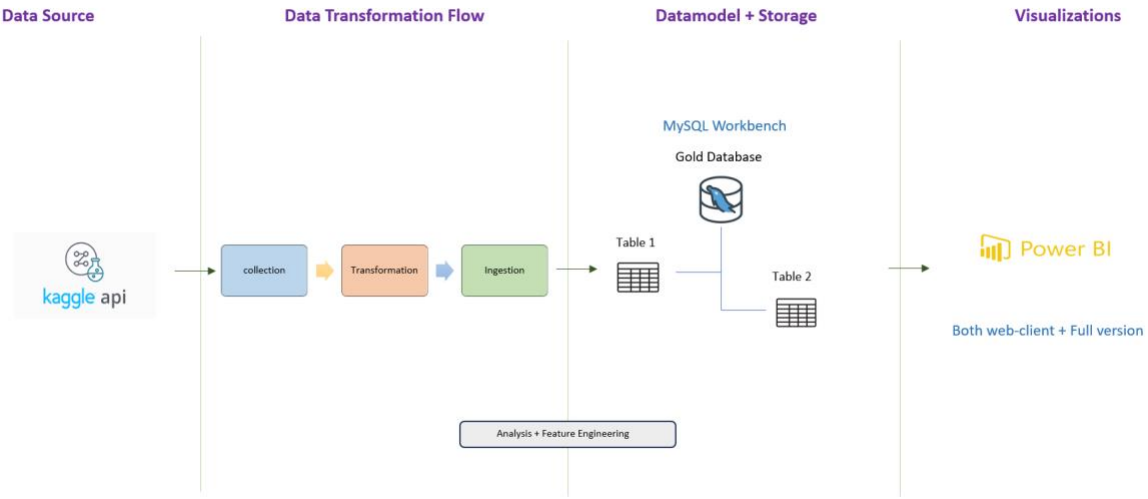
2. Objective

The objective of this project is to design and implement a fully automated ETL pipeline that handles data from initial extraction to final ingestion with minimal manual intervention. The project aims to provide a robust solution capable of collecting raw datasets, transforming them through systematic cleaning and restructuring, and loading them into a MySQL database. Additionally, the goal is to create a CI/CD-enabled system that ensures that every pipeline run executes consistently, reliably, and in a version-controlled environment.

3. Architecture Diagram

The architecture typically includes:

- Data Source Layer
- Collection Scripts
- Transformation Engine
- Storage Layer (MySQL)



4. Project Structure

The project is structured into multiple folders, each serving a specific purpose within the ETL lifecycle. This design ensures modularity, cleanliness, and clear separation of responsibilities.

• The config/ directory contains configuration files including database connection details and SQL schema definitions. These files allow parameterization and reusability without modifying core scripts.

• The data/raw/ folder stores unprocessed datasets exactly as they were collected. This ensures data lineage and traceability. No modifications are made at this stage.

• The data/processed/ folder includes intermediate datasets that have undergone partial cleaning or restructuring. These serve as checkpoints during transformations.

• The data/gold/ folder contains the final, fully transformed output datasets in user-friendly formats such as Excel or JSON. These cleaned datasets are ready for ingestion or visualization.

• The src/collect/ directory includes Python scripts dedicated to fetching or downloading raw data. This layer isolates extraction logic from other operations.

• The src/transform/ directory stores transformation scripts responsible for cleaning, validating, and restructuring raw data into usable formats. This step ensures adherence to quality and business rules.

• The src/ingest/ directory contains scripts that load gold-layer data into a MySQL database. It ensures schema alignment and performs insertion with error handling.

6. Advantages

The project offers several advantages:

• Automation reduces human intervention, lowering the chance of errors and ensuring consistent execution.

• Modularity allows components to be updated independently without affecting the entire system.

• Separation into raw, processed, and gold layers ensures data governance, traceability, and better debugging.

• CI/CD integration helps maintain continuous processing and enables complete visibility of all pipeline runs.

• Integration with MySQL provides structured storage ready for analytics tools such as Power BI and Spotfire.

7. Limitations

Despite its strengths, the project has certain limitations:

• The pipeline depends on the availability and stability of external raw data sources. Any change or delay in source datasets directly affects the workflow.

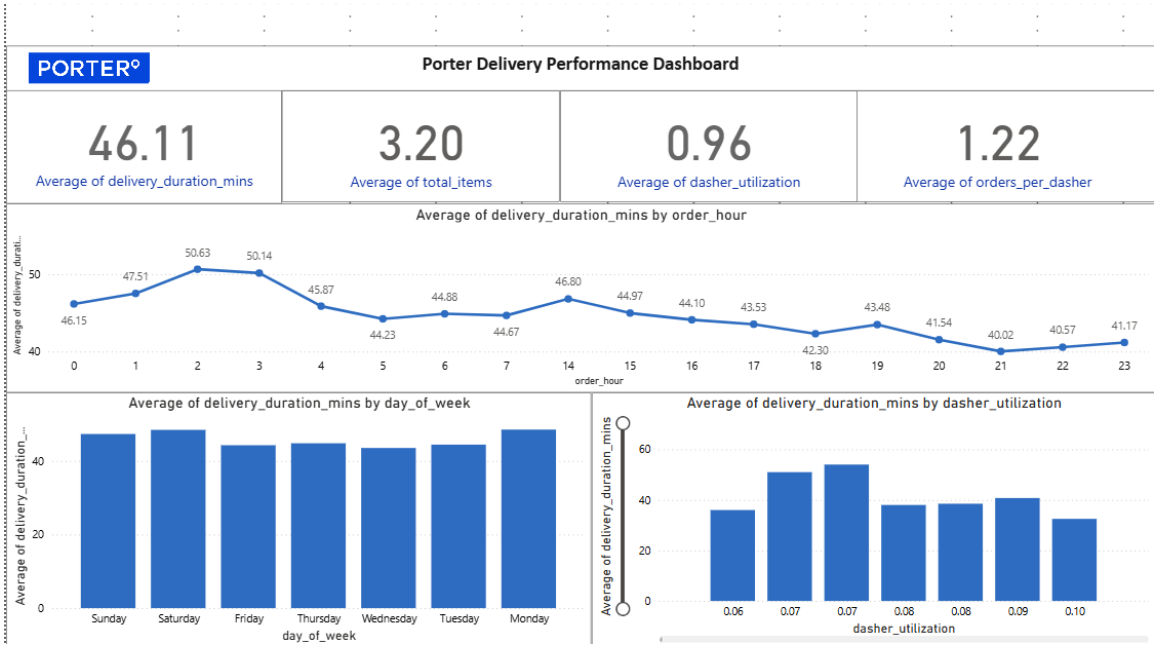
• Initial setup requires configuring MySQL, environment variables, and dependency installation, which may require technical expertise.

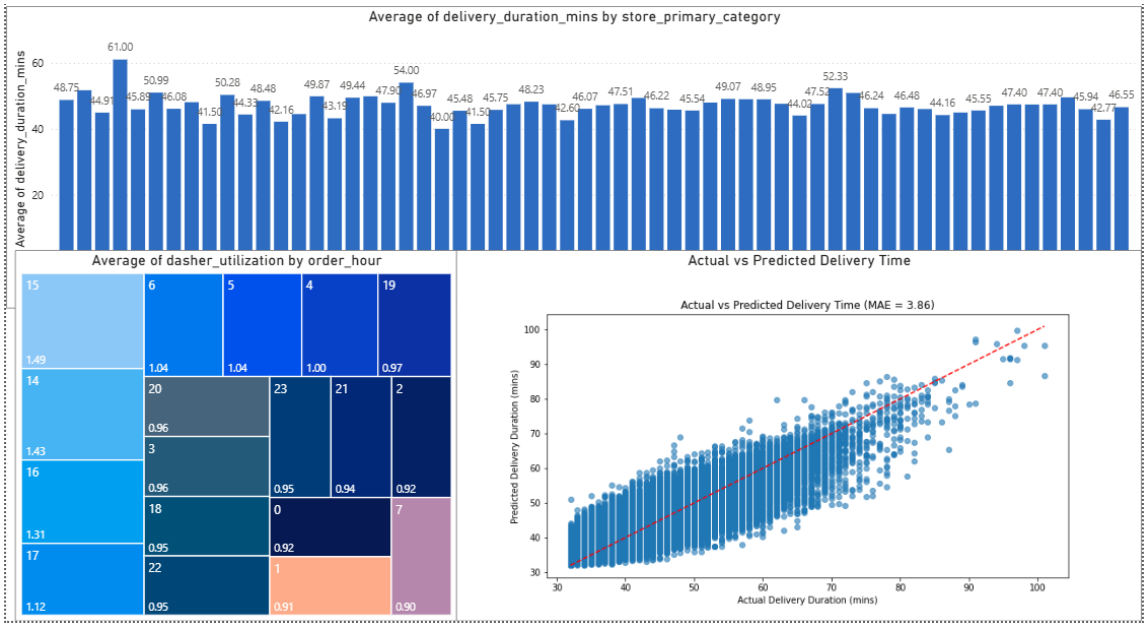
• GitHub Actions requires internet access and cannot be executed offline.

• The current version lacks advanced logging, monitoring, and alerting mechanisms. Future improvements can include dashboards or alerts for errors and performance metrics.

8. Final Output

Power BI Report





9. References

- Python Documentation : https://docs.python.org/3/
- Pandas and OpenPyXL Libraries: https://pandas.pydata.org/
- MySQL Documentation : https://www.mysql.com/
- GitHub Actions Documentation : https://docs.github.com/en
- SQL Schema Design Resources : https://docs.ejabberd.im/developer/sql-schema/